Introduction to Agile Software Development

Agile Software Development

Iterative, collaborative, and adaptive approach to software creation.

- Customer Collaboration
- Working Software
- Responding to Change
- Individuals & Interactions

Dr. Mohamed H. ElGazzar

Dr. Mohamed ElGazzar
e14160@eng.asu.edu.eg

Grade Distribution

- 20 Mark mid term
- 40 Final exam
- 2 Quizzes – total 10 marks
- Assignments/Lab 10 marks
- Project Class work 20 marks

Course topics

- Overview On Software Models
- Comparison between Agile and other software models
- Case study for different software models
- Principles of Agile
- Scrum framework
- Scrum roles
- Requirements and user stories
- Product backlog
- Sprints
- Estimation and velocity
- Sprint planning
- Spring Execution
- Sprint Retrospective
- Agile Testing

Software is Engineered, Not Just Written

Moving Beyond Code Generation to System Lifecycle Management

The Craft of
Writing Code

The Discipline of
Engineering Systems

From Coding to Engineering: Key Distinctions

Writing Code
(The Activity)

Focus: Immediate functionality & individual output.

Tools: Text editors, compilers, basic debuggers.

Perspective: Short-term, task-oriented completion.

Quality: Ad-hoc testing, relying on intuition.

Software Engineering
(The Discipline)

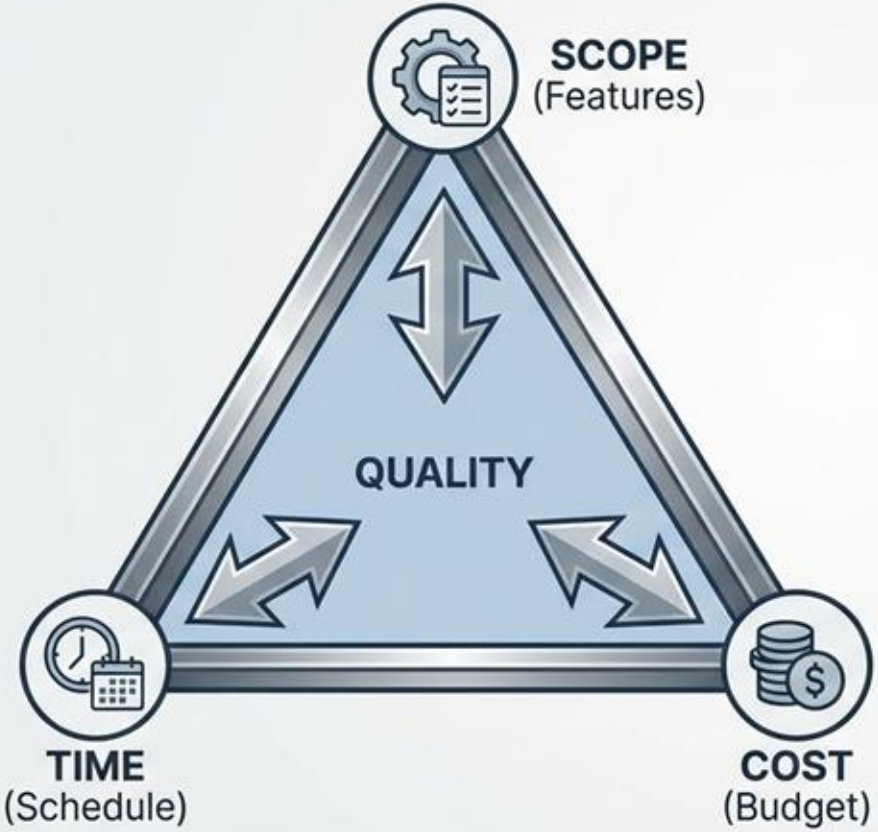
Focus: System reliability, scalability, & maintainability.

Tools: IDEs, CI/CD, version control, automated testing.

Perspective: Long-term product lifecycle & evolution.

Quality: Rigorous, automated testing & formal verification.

The Project Management "Iron Triangle"



Key Principles:

1. Every project balances these three constraints.
2. You generally can't fix all three without compromising quality.
3. Changing one constraint inevitably affects one or both of the others.

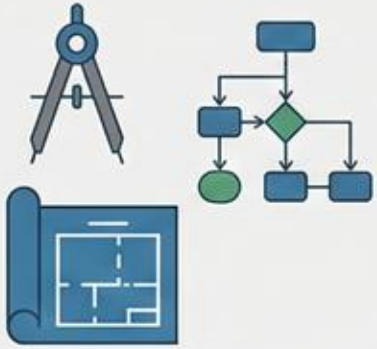
For example: Increasing Scope typically requires more Time and/or Cost, or Quality will suffer.

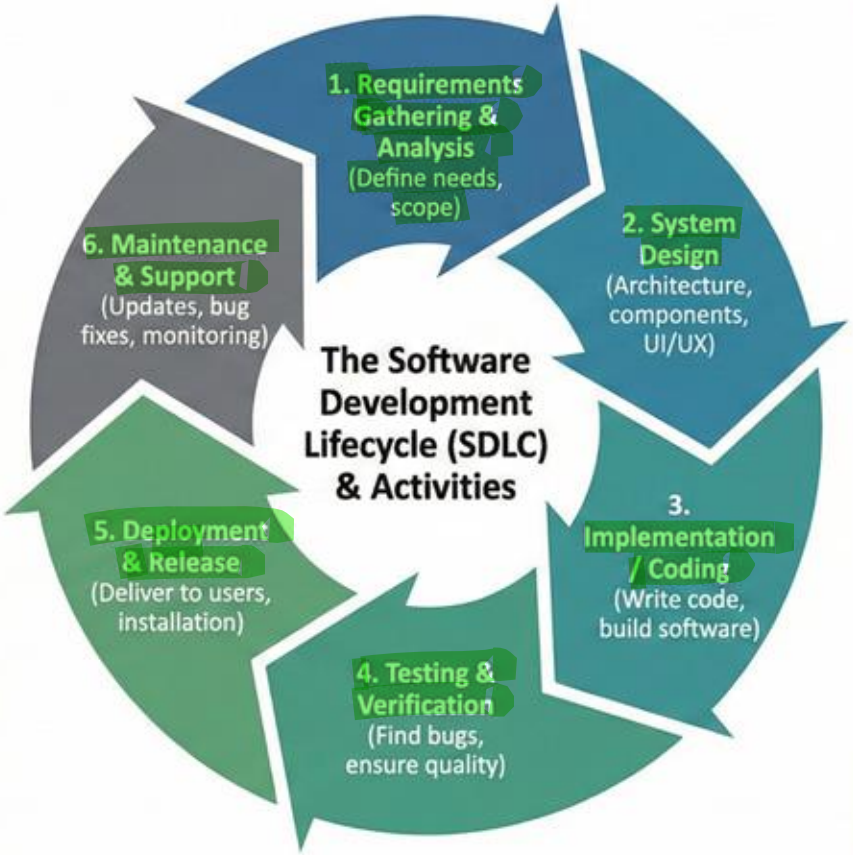
Software Model & Associated Activities: A Framework for Development

What is a Software Model?

An abstract representation of a software process. A roadmap that guides the entire lifecycle from conception to retirement, outlining phases, tasks, and deliverables.

- e.g., Waterfall, Agile, Scrum, DevOps, Spiral, V-Model.





Software Process Descriptions: Definition & Purpose

What is a Software Process Description?

A formal documentation that outlines the structure, steps, roles, and artifacts of a software development process.

- Provides a roadmap for the project team.
- Ensures consistency and repeatability.
- Facilitates communication and collaboration.
- Enables process improvement and optimization.

Key Components

- **Activities:** Defined tasks and actions.
- **Roles:** Responsibilities and accountability.
- **Artifacts:** Deliverables and documentation.
- **Tools:** Software and methodologies used.
- **Schedule:** Timeline and milestones.

Software process descriptions are essential for managing complexity and achieving project goals.

Risk Management in Software Models

Different models as strategies for managing product and technical risks.

Product Risk:
Building the Wrong Thing

Mitigated by: Upfront planning, user feedback, iterative prototypes.

Technical Risk:
Building it Poorly

Mitigated by: Testing, code reviews, automation, architectural standards.

Waterfall (Upfront Planning)

Focuses on detailed requirements to reduce product risk early on (can be inflexible).

Agile (Iterative Feedback)

Addresses both risks through continuous user involvement and technical excellence in each iteration.

DevOps (Continuous Delivery)

Emphasizes automation and monitoring to minimize technical risk and ensure stability.

No "Silver Bullet" in Software Models

There is no single perfect methodology. The "best" choice is always context-dependent.

The Myth of the Universal Solution

No model is without trade-offs. Every approach has its strengths and weaknesses.

The Right Choice Depends On:

Project Type
Complexity, duration, and risk level.

Team Size
Number of members, location, and expertise.

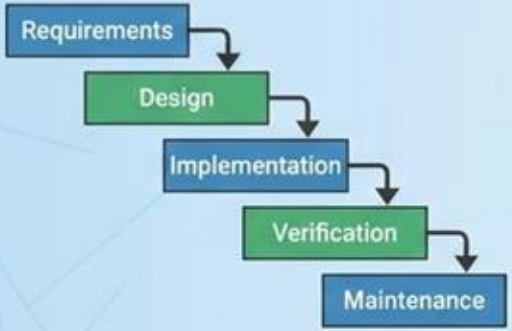
Context
Complexity, duration, and expertise.

Requirements Clarity
How well-defined and stable the goals are.

What are Software Development Models?

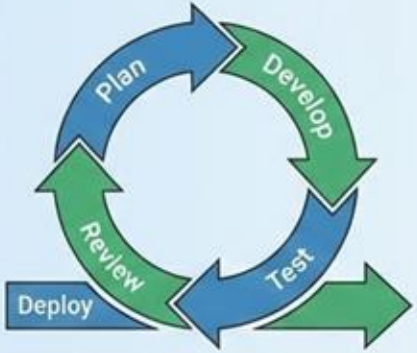
Frameworks that provide structure to the software creation process, defining activities, steps, and deliverables.

Waterfall Model



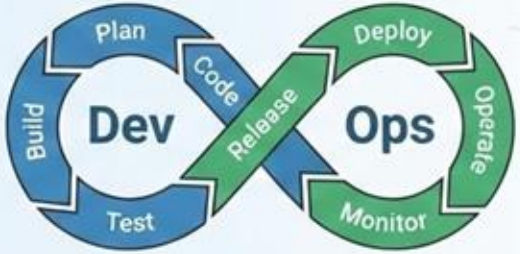
Linear, sequential phases. Each phase must be completed before the next begins.

Agile Model



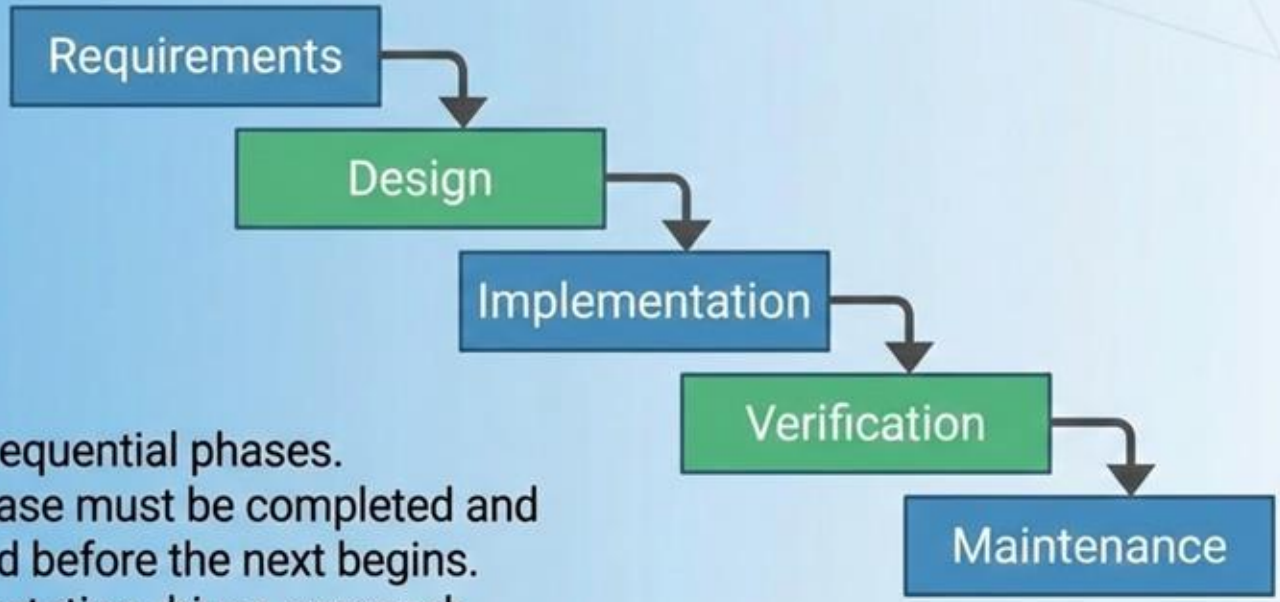
Iterative, incremental, and collaborative approach with continuous feedback.

DevOps Model



Combines development and operations for continuous delivery and integration.

What is the Waterfall Model?



- Linear, sequential phases.
- Each phase must be completed and approved before the next begins.
- Documentation-driven approach.
- Best suited for projects with well-defined, stable requirements.
- Little flexibility for changes once a phase is complete.

Waterfall Model: Requirements Phase

This is the initial phase where all system requirements are gathered and documented in detail.

- **Activities**: Stakeholder interviews, requirement analysis, feasibility study, documentation.
- **Deliverable**: Software Requirements Specification (SRS) document.

Waterfall Model: Design Phase

In this phase, the system and software design specifications are created based on the requirements.

- **Activities**: System architecture design, database design, UI design, component design.
- **Deliverable**: System Design Document (SDD), high-level and low-level design documents.

Waterfall Model: Implementation Phase

This is where the actual coding and building of the software takes place.

- **Activities**: Writing code, unit testing, code reviews, integration.
- **Deliverable**: The developed software application.

Waterfall Model: Verification Phase

The developed software is rigorously tested to ensure it meets all specified requirements.

- **Activities**: System testing, user acceptance testing (UAT), bug fixing.
- **Deliverable**: Tested and verified software, test reports.

Waterfall Model: Maintenance Phase

This final phase involves ongoing support and modifications after the software is deployed.

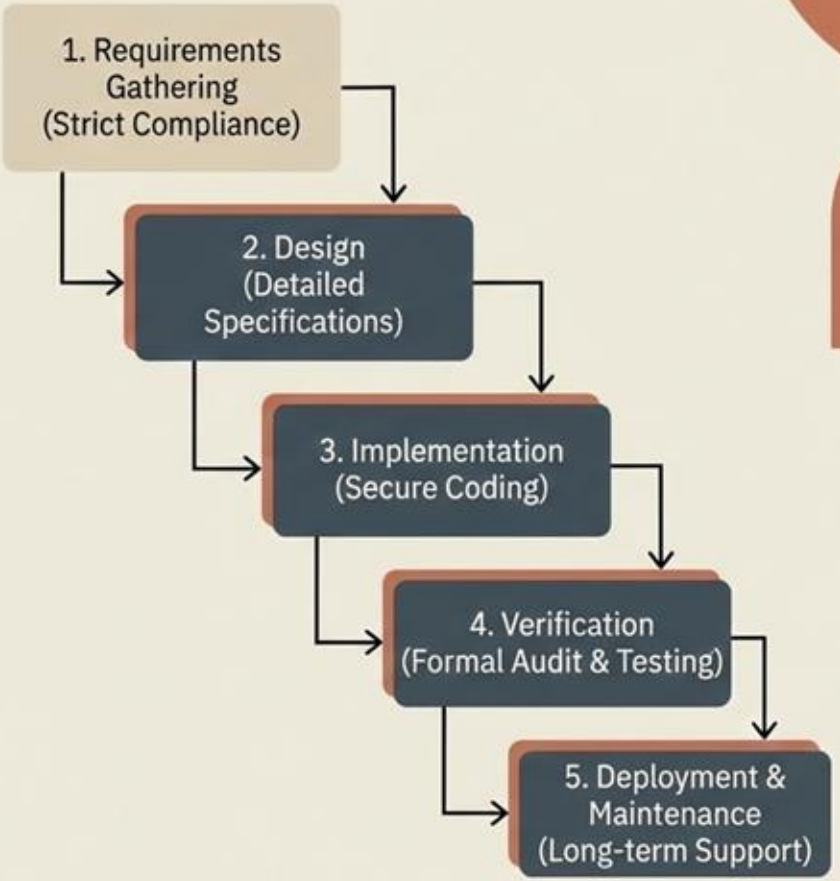
- **Activities**: Bug fixes, updates, performance enhancements, porting to new platforms.
- **Deliverable**: Updated and maintained software.

Waterfall Model: When to Use It

- Requirements are well-understood, documented, and stable.
- The project is short and the product definition is stable.
- Technology is understood and not dynamic.
- There are strict regulatory or compliance requirements.
- Resources with required expertise are available freely.

Waterfall Model Use Case: Government Regulatory Software Software

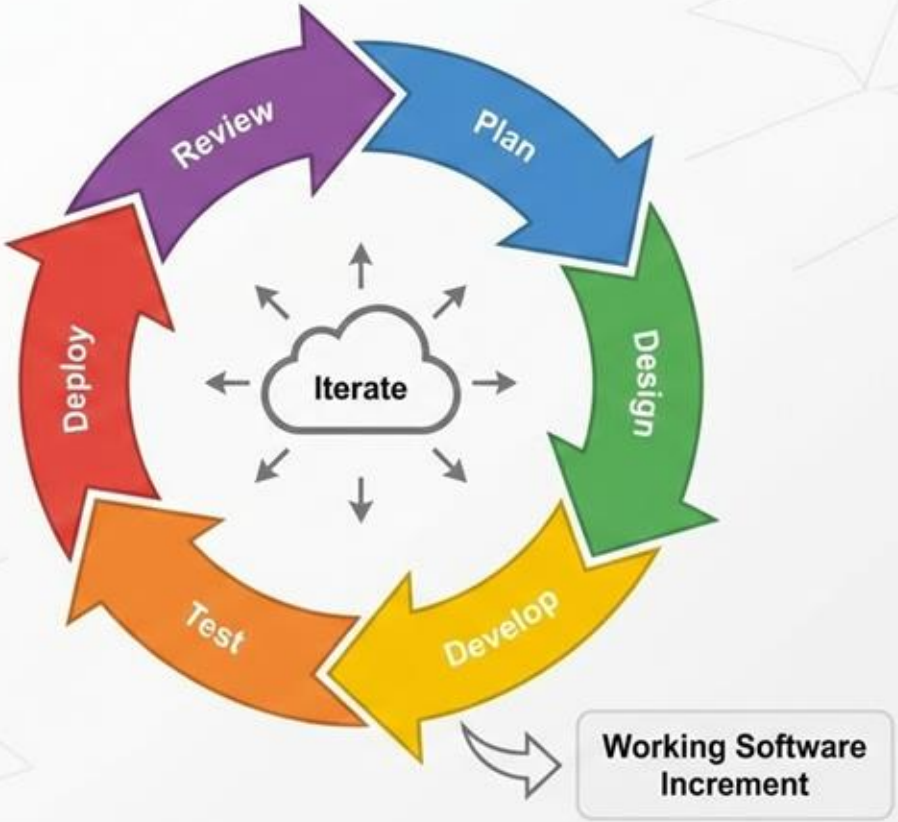
Example: Developing a secure, complaint-management system for a government agency:



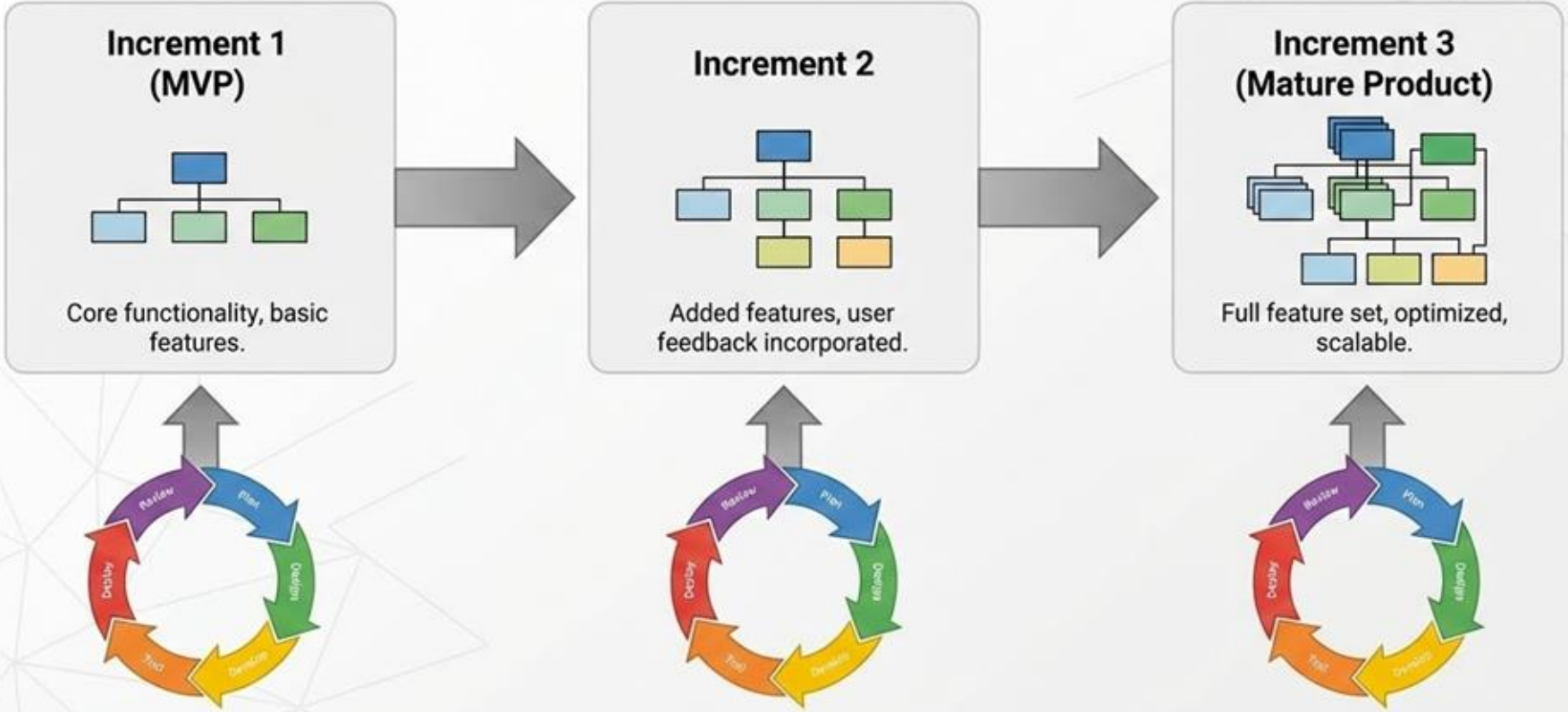
What is Agile?

- An iterative and incremental approach to software development.
- Focuses on flexibility, collaboration, and customer feedback.
- Delivers working software in small, frequent releases.
- Adapts to changing requirements throughout the project.

Agile Software Development Model Diagram

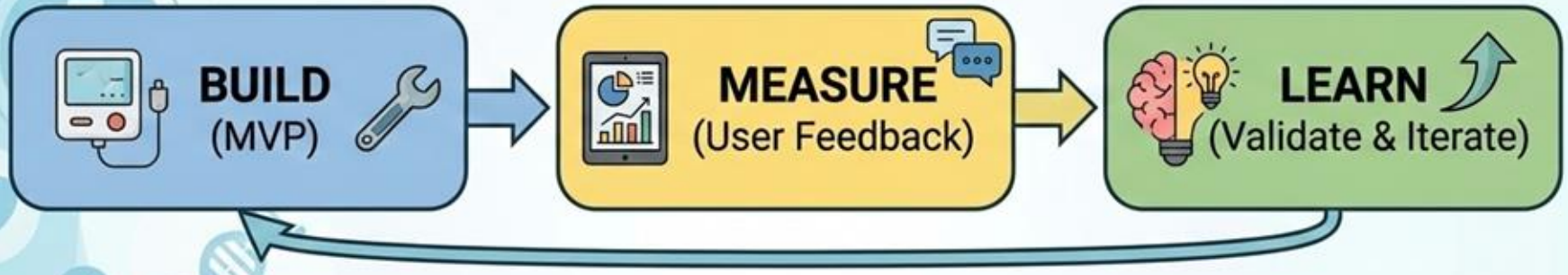


Agile Software Development: Progressive Increments



What is a Minimum Viable Product (MVP) in Agile?

A product version with just enough core features to be usable by early customers and provide critical feedback for future development.



Key Characteristics & Benefits

- **Focus on Core Value:** Solves the primary problem with minimal features.
- **Faster Time-to-Market & Reduced Risk:** Validates assumptions early with less investment.
- **Early & Continuous Learning:** Gathers real-world insights to guide future iterations.

Agile Software Development: The 6 Phases

1. Plan → **2. Design** → **3. Develop** → **4. Test** → **5. Deploy** → **6. Review**

1. Plan	2. Design	3. Develop	4. Test	5. Deploy	6. Review
Define scope, gather requirements, prioritize tasks for the iteration.	Create architecture, UI/UX, and technical specifications.	Write code, build features, and perform unit testing.	Verify functionality, identify bugs, ensure quality.	Release the working software increment to a staging or production environment.	Gather stakeholder feedback, reflect on the process, and identify improvements.

*These phases are repeated in each iteration, building upon the previous increment, as shown in the previous slides.

When to Use Agile Software Development

Unclear or Evolving Requirements

When the project scope is not fully defined and is expected to change, allowing for flexibility.

Need for Rapid Delivery

When a working prototype or MVP is needed quickly to gather user feedback and enter the market.

Continuous Customer Feedback is Crucial

When direct and ongoing input from end-users is required to guide the development process.

Complex Projects with High Uncertainty

When breaking down a large, complex problem into smaller, manageable, and testable increments is beneficial.

Small, Collaborative, and Cross-functional Teams

When the development team is self-organizing, sits together, and requires high levels of daily communication.

Example Use Case: 'FitLife' Mobile App Startup

The Challenge: Unclear User Needs

A startup wants to build a fitness app but doesn't know which features (tracking, social, coaching) users will value most.

The Goal: Rapid Market Entry

They need to launch a Minimum Viable Product (MVP) quickly to gain a competitive advantage and secure funding.

The Agile Approach: Iterative Feedback

The team releases a basic MVP, gathers user feedback via in-app surveys, and prioritizes new features for the next 2-week sprint.

The Result: Reduced Risk & Wasted Effort

Instead of building a massive, unwanted app, they build only what users actually need, adapting to market changes.

The Team: Cross-functional Collaboration

Developers, designers, and product owners work closely together, holding daily stand-ups to align on goals.

Agile Use Case: 'FoodieFinds' Restaurant Recommendation App

The Scenario: Building a New Restaurant Discovery App

Startup with a new concept, unsure of user preferences, facing a rapidly changing market with competitors.

Why Agile is Appropriate: Flexibility & Speed

Allows for quick release of an MVP to test core assumptions, enables rapid iteration based on real user feedback, and facilitates pivoting if necessary.

Why Waterfall is Not Suitable: Rigidity & Delayed Feedback

Requires upfront, detailed requirements which are impossible to know. A single, long development cycle risks building the wrong product and delays crucial market feedback.

What is the DevOps Model?

A cultural shift and set of practices that emphasizes collaboration and communication between software development (Dev) and IT operations (Ops) teams. It aims to shorten the systems development life cycle and provide continuous delivery with high software quality.

Detailed Phases of the DevOps Lifecycle

Development Phases (Dev)

PLAN: Define requirements, create backlog, manage project.

CODE: Write software, use version control.

BUILD: Compile code, create artifacts, run initial checks.

TEST: Automated and manual testing for quality and bugs.

Operations Phases (Ops)

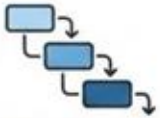
RELEASE: Package software, manage release schedule.

DEPLOY: Push to production or staging environments.

OPERATE: Manage and maintain running application and infrastructure.

MONITOR: Collect data, track performance, gather user feedback.

Requirements Engineering: Waterfall vs. Agile

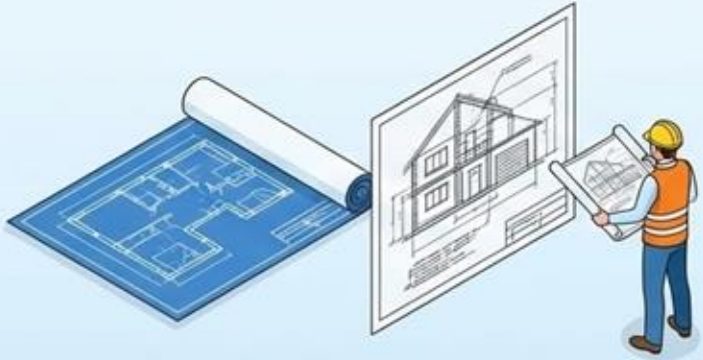


Waterfall Model	Agile Model
Upfront, comprehensive gathering.	Iterative, continuous discovery.
Detailed, formal documentation (SRS).	Lightweight documentation (User Stories, Epics).
Fixed scope, change is difficult.	Evolving scope, embraces change.
Sign-off and baselining before development.	Constant feedback and refinement.
Emphasis on completeness and clarity.	Emphasis on collaboration and adaptability.

Design & Implementation: Waterfall vs. Agile

Waterfall Model

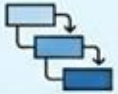
- Sequential, phase-based approach.
- Complete, detailed design *before* coding.
- Big Design Up Front (BDUF).
- Implementation is a long, single phase.
- Testing after implementation completes.



Agile Model

- Iterative and incremental approach.
- Design and implementation are concurrent.
- Emergent design, evolves with product.
- Continuous integration and testing.
- Working software delivered frequently.

Testing in Waterfall vs. Agile



Waterfall Model

- Testing is a distinct, separate phase.
- Occurs only after implementation is complete.
- Heavy reliance on formal test plans and documentation.
- Defects found late, expensive to fix.
- Focus on verification against requirements.

Agile Model

- Testing is continuous and concurrent.
- Automated testing (unit, integration) is essential.
- Emphasizes working software over documentation.
- Defects found early, cheap to fix.
- Focus on validation and user feedback.

Contracts: Waterfall vs. Agile

Waterfall Contract	Agile Contract
Fixed-price, fixed-scope contracts common.	Time and materials (T&M) or capped T&M.
Detailed specifications required upfront.	Scope is flexible, evolves with project.
Changes managed via formal change requests (costly).	Emphasizes collaboration and trust.
Risk primarily on vendor for delivery.	Risk is shared; customer prioritizes, vendor delivers.
Focus on adherence to initial plan and budget.	Focus on value delivery and adaptability.

Tools Used in Agile Software Development Phases

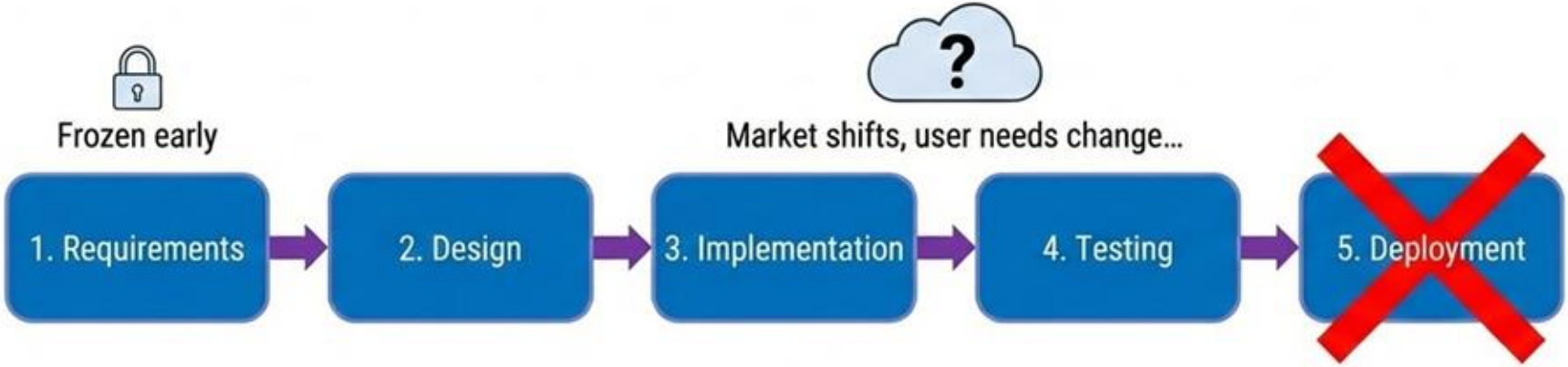
Agile Phase	Key Tools	Primary Function
Plan	Jira, Trello, Asana	Project management, backlog tracking, sprint planning.
Develop	Git, GitHub, VS Code, IntelliJ IDEA	Version control, coding, integrated development environments.
Test	Selenium, JUnit, TestRail, Cucumber	Automated & manual testing, bug reporting, quality assurance.
Deploy	Jenkins, Docker, Kubernetes, Ansible	Continuous Integration/Continuous Deployment (CI/CD), containerization, orchestration.
Review	Slack, Zoom, Microsoft Teams, Mural	Collaboration, communication, sprint review & retrospective meetings.
Launch	AWS, Azure, Google Cloud Platform (GCP)	Cloud infrastructure, hosting, release management.

Case Study: Project X – Choosing the Right Model

Developing a Next-Gen Mobile Banking App

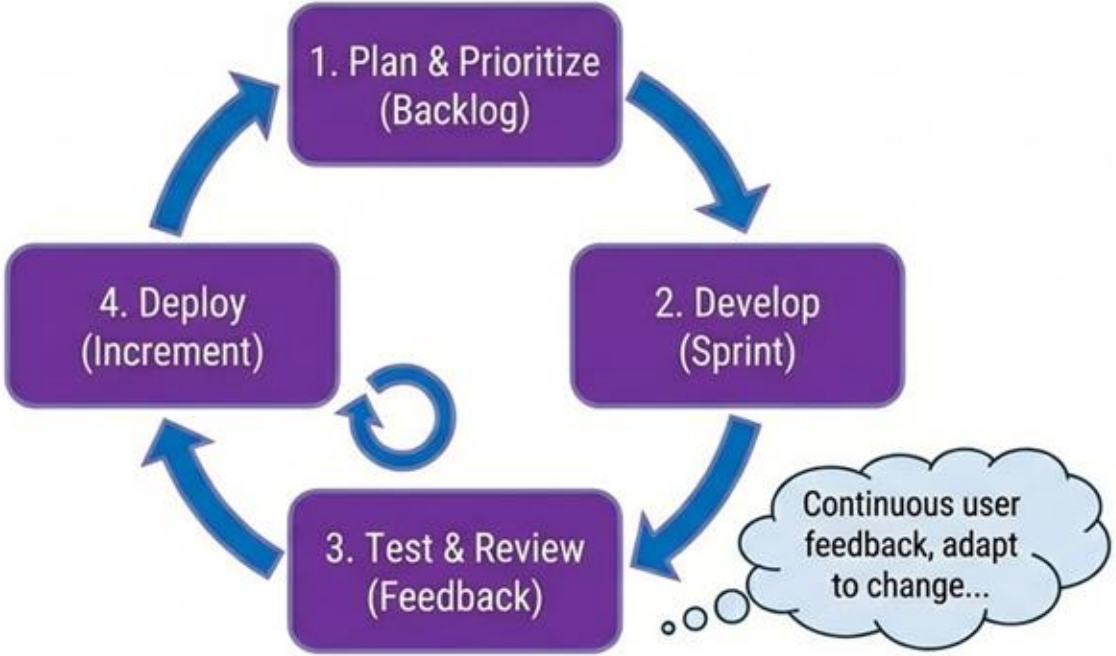
- **The Challenge:** Deliver a competitive, user-friendly app in a rapidly evolving market.
- **The Question:** Which development model—Agile or Waterfall—is best suited for success?

Scenario A: The Waterfall Approach



Result: Late delivery, outdated features, high risk of failure.

Scenario B: The Agile Approach



Result: Early value, adaptable to change, lower risk.

Conclusion: When to Use Each Model

Waterfall is Best When...	Agile is Best When...
• Requirements are well-defined and stable. • Strict regulatory compliance needs. • Predictable timeline and budget are paramount. • Low uncertainty, high predictability.	• Requirements are uncertain or likely to change. • Speed to market (MVP) is critical. • Continuous customer feedback is valued. • High uncertainty, innovation focus.

Key Takeaway: Choose the model that best aligns with your project's uncertainty and need for adaptability.

Case Study: Choosing the Right Software Development Model

Focusing on When to Use Waterfall over Agile

- Exploring two distinct scenarios
- Analyzing project characteristics
- Determining the optimal approach

Healthcare Case Study: Medical Imaging System Upgrade

A strategic choice for Waterfall over Agile

Waterfall Model

↓

Successful Implementation

Focus: Compliance, Safety, Predictability

The Challenge: Why Agile Failed in a Critical System

- Evolving requirements clashed with strict regulatory compliance.
- Frequent iterations caused re-validation bottlenecks and delays.
- Lack of a fixed final state led to integration issues with legacy systems.
- Unpredictable timelines and budgets became unacceptable.

The Solution: A Structured Waterfall Approach

1. Requirements & Regulatory Analysis (Frozen Scope)

Comprehensive definition & sign-off

2. Detailed System Design & Architecture

Blueprint for all components

3. Rigorous Development & Unit Testing

Building to specification

4. System Integration, Validation & Deployment

Final testing & release

Conclusion: When to Use Waterfall (and Avoid Agile) in Healthcare

Waterfall is Best When:	Agile is Less Suitable When:
✓ High regulatory compliance is mandatory. ✓ Requirements are fixed and well-defined. ✓ Predictability of cost and timeline is paramount. ✓ Project is safety-critical with zero tolerance for error.	✕ Scope is rigid and cannot be changed. ✕ Continuous validation is impractical or costly. ✕ Project has a high cost of failure (e.g., patient safety). ✕ Stakeholders cannot commit to frequent engagement.

Waterfall provides the necessary structure and control for safety-critical, compliance-heavy projects.